
Lab 06

Objective:

The objective of this lab is to understand Keras – a commonly used python library for implementing artificial neural networks. In this lab, the students will learn how to implement ANNs in python using Keras library.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement ANN in Keras..
- Train ANN using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://www.educative.io/answers/how-to-build-a-simple-neural-network-using-keras>
2. <https://towardsdatascience.com/building-our-first-neural-network-in-keras-bdc8abbc17f5>

14) Useful Concepts

- **Keras:** Keras is a high-level, user-friendly API used for building and training neural networks. It is an open-source library built in Python that runs on top of TensorFlow. It was developed to enable fast experimentation and iteration, and it lowers the barrier to entry for working with deep learning. In this lab, we'll see how to install and start using Keras; the Sequential API; and the steps for building, compiling, and training a model.
- **Installing Keras:** To start with, you'll want to install TensorFlow. Depending on which operating system you're using, this might look slightly different, but for the most part, you can use pip, Python's package manager:

```
# First upgrade pip
pip install --upgrade pip

# Then install TensorFlow
pip install tensorflow
```

Once you've installed TensorFlow, all you need to do to use Keras is to run the following import statement at the top of your script or notebook:

```
from tensorflow import keras
```

- **Keras' Sequential API:** The Sequential API is the easiest way to use Keras to build a neural network. Using the Sequential class, it's possible to stack a variety of different layer types, one after the other, to produce a neural network. There are many types of layers available in the Keras Sequential API. One of the most common layer types is the Dense layer, a fully connected layer, but there are many others:
 - **Convolutional layer:** a layer for processing images — used for convolutional neural networks.
 - **Recurrent layer:** a layer for processing sequences of data — used for recurrent neural networks.
 - **MaxPooling layer:** a layer for down-sampling feature maps by taking the maximum value in non-overlapping rectangular blocks — used for preserving important features while reducing the chance of overfitting.
 - **Flatten layer:** a layer that flattens the multi-dimensional input tensors into a single dimension — used as a transition layer between convolutional or recurrent layers and fully connected layers in a neural network.

- **Dropout layer:** a layer that randomly sets input units to 0 (using a defined frequency) during training time — used as a regularization technique to prevent overfitting in neural networks.
- **Embedding layer:** a layer that represents words or phrases in a high-dimensional vector space — used to map words or phrases to dense vectors for use as input to a neural network.

These are just a few examples of the many types of layers available in the Keras Sequential API. Each layer is designed to perform a specific type of computation on the inputs, and they can be combined to create powerful neural network architectures.

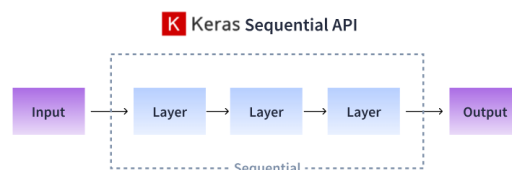
- **Building a Model with Keras:** To build a model with the Keras Sequential API, the first step is to import the required class and instantiate a model using this class:

```
from tf.keras import Sequential  
  
model = Sequential()
```

Next, choose the layer types you wish to include, and add them one at a time to the sequential model you've instantiated. For example, to add three Dense hidden layers and one output layer to an input layer, your code might look like this:

```
from tf.keras import Dense  
  
model.add(Dense(input_shape=(16,)))  
model.add(Dense(8))  
model.add(Dense(4))  
model.add(Dense(2))  
model.add(Dense(1))
```

In this example, we've added an implicit input layer with 16 nodes and three hidden layers with 8, 4, and 2 nodes, respectively. Finally, we've added an output layer with just one node.



-
- **Compiling a Model with Keras:** Once you've built a model using the Keras Sequential API, you'll need to compile it before it can be used for training. When compiling the model, you'll need to specify both a loss function and an optimizer. There are many different options to choose from:

Loss functions:

- Mean Squared Error (MSE): a common loss function for regression problems — measures the average squared difference between the predicted and actual values.
- Binary Crossentropy: a loss function for binary classification problems — measures the cross-entropy between the predicted and actual binary distributions.
- Categorical Crossentropy: a loss function for multi-class classification problems — measures the cross-entropy between the predicted and actual categorical distributions.

Optimizers:

- Stochastic Gradient Descent (SGD): a simple optimization algorithm that updates the parameters by computing the gradient of the loss function with respect to the parameters.
- Adam: an optimization algorithm that adapts the learning rate based on the historical gradient information.
- RMSprop: an optimization algorithm that uses a moving average of squared gradients to normalize the gradient updates.

These are just a few examples of the many loss functions and optimizers available in Keras. The choice of loss function and optimizer will depend on the specific problem you're trying to solve and the characteristics of your data.

For example, to compile a model with Mean Squared Error as the loss function and Adam as the optimizer, we would use the following code:

```
model.compile(loss='mean_squared_error', optimizer='adam')
```

- **Fitting a Model with Keras:** Finally, to use Keras for deep learning, the compiled model must be fit to a training dataset. The training dataset should be prepared using a process that separates the independent variables, the features (or X variable) from the dependent variable, the target (or y variable). These variables should also be converted to NumPy arrays and reshaped as needed, and the `input_shape` parameter of the neural network will

need to be configured to match the training data shape. Once all of these preprocessing steps are in place, you can simply fit the model to the training data like so:

```
model.fit(x_train, y_train)
```

To evaluate the performance of the model after training, you can use evaluate:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

15) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>30</i>	<i>Medium</i>	<i>CLO-3</i>
<i>2</i>	<i>30</i>	<i>High</i>	<i>CLO-3</i>

Activity 1:

Students will learn to implement an artificial neural network (ANN) using Keras on a dataset. They will conduct experiments to understand the impact of various hyperparameters and model architectures on the performance of the ANN.

Solution:

Building the Neural Network:

```
# Import necessary libraries
```

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Flatten
```

```
from tensorflow.keras.utils import to_categorical
```

```
from tensorflow.keras.datasets import mnist
```

```
# Load and preprocess the dataset
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

```
# One-hot encode the labels

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)


# Build the neural network model

model = Sequential([

    Flatten(input_shape=(28, 28)), # Flatten the input images to 1D vector

    Dense(128, activation='relu'), # First hidden layer with 128 neurons and ReLU activation

    Dense(64, activation='relu'), # Second hidden layer with 64 neurons and ReLU activation

    Dense(10, activation='softmax') # Output layer with 10 neurons (one for each class) and softmax
    activation

])


# Compile the model

model.compile(optimizer='adam',

              loss='categorical_crossentropy',

              metrics=['accuracy'])


# Train the model

model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)


# Evaluate the model

test_loss, test_acc = model.evaluate(x_test, y_test)
```

```
print(f'Test accuracy: {test_acc:.4f}')
```

Activity 2: Experiments

- **Experiment 1:** Change the number of neurons in the hidden layers and observe the effect on accuracy.

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(256, activation='relu'), # Changed to 256 neurons
    Dense(128, activation='relu'), # Changed to 128 neurons
    Dense(10, activation='softmax')
])
```

- **Experiment 2:** Change the activation function to `sigmoid` and observe the effect on accuracy.

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='sigmoid'), # Changed to sigmoid
    Dense(64, activation='sigmoid'), # Changed to sigmoid
    Dense(10, activation='softmax')
])
```

Experiment 3: Change the optimizer to `sgd` and observe the effect on accuracy

```
model.compile(optimizer='sgd',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Experiment 4: Change the number of epochs to 20 and observe the effect on accuracy.

```
model.fit(x_train, y_train, epochs=20, batch_size=32, validation_split=0.2)
```

16) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement artificial neural networks in Keras. The student will also learn how to perform experiments.

Lab Task 1: Building and Training a Neural Network Using Keras

Students will build and train a neural network to predict wine quality using the provided dataset. They will preprocess the data, create a neural network model, train it, and evaluate its performance. For this task, use the "Wine Quality" dataset from the UCI Machine Learning repository: [Wine Quality Data Set](#).

Instructions for Submission:

7. Submit a Jupyter Notebook or Python script with the complete implementation.
8. Include comments in the code to explain each step.
9. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results

Lab Task 2: Enhancing Neural Network Performance Using Keras

Students will demonstrate their ability to improve the performance of a neural network using Keras on a dataset from the Kaggle or UCI Machine Learning repository. They will learn to apply techniques such as hyperparameter tuning, regularization, and advanced optimizers to enhance model performance.

Instructions: Apply following improvement techniques

- Hyperparameter Tuning: Experiment with different numbers of neurons, layers, epochs, and batch sizes.
- Advanced Optimizers: Experiment with different optimizers like RMSprop or SGD with momentum.

Instructions for Submission:

1. Submit a Jupyter Notebook or Python script with the complete implementation.
2. Include comments in the code to explain each step.
3. Provide a brief report (1-2 pages) summarizing the following:
 - Initial model performance
 - Techniques applied for improvement
 - Improved model performance
 - Observations and insights from the results